Lecture 5

Andrei Arsene Simion

Columbia University

September 30, 2025

Outline

- Optimization - Theory + Examples
- Residual Connections
- BatchNorm and LayerNorm
- Regularization - Dropout

Outline

- Optimization - Theory + Examples
- Residual Connections
- BatchNorm and LayerNorm
- Regularization - Dropout

Optimization

- We usually have a problem like $\min_\theta(\mathcal{L}(\theta))$ where $\mathcal{L}$ is some loss function
- $\mathcal{L}$ usually depends on your problem and on the training data
  - $\mathcal{L}(\theta) = |y - \theta|$
  - $\mathcal{L}(\theta) = \frac{\sum_{i=1}^{N}(y^{(i)} - \theta_0 - \theta_1 x^{(i)})^2}{N}$
  - $\mathcal{L}(\theta) = -\frac{\sum_{i=1}^{N}\sum_{k=1}^{K} y_k^{(i)} \log a_k^{(i)}}{N}$
- More complicated examples: Skip-Gram, CBOW, CNN
- There are recipes we use to optimize these problems

Optimization

- An example of gradient descent
- Issue 1: We want gradient updates to be smooth, so we make progress
- Issue 2: We want to balance the way we move, even if a gradient is larger with respect to one dimension vs another

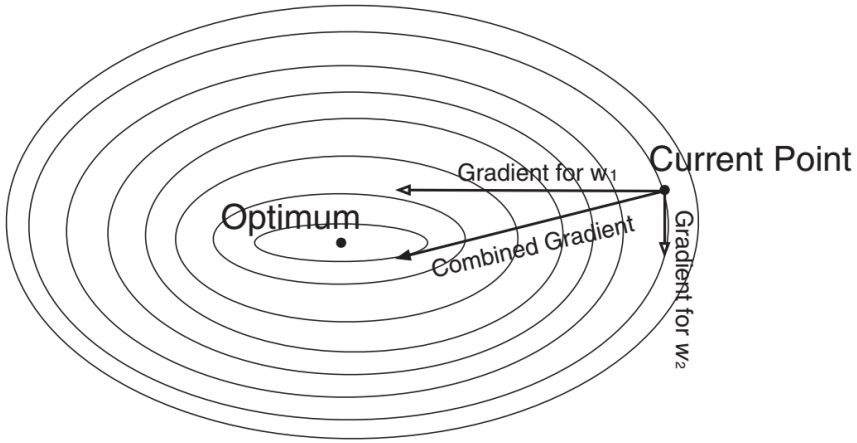


Figure 5.5 Gradient descent training. We compute the gradient in regard to every dimension. In this case the gradient with respect to weight w_2 is smaller than the gradient with respect to the weight w_1, so we move more to the left than down (note: arrows point in negative gradient direction, pointing to the minimum).

SGD

- Stochastic Gradient Descent is the most popular method to optimize $\min_\theta(\mathcal{L}(\theta))$
- Divide your training set into batches $B_1, \ldots, B_T$ which cover your entire data
- Compute the gradient of $\mathcal{L}$ on these batches (i.e. assume the full data is just batch B_t and consider $\mathcal{L}_{B_t}$ and optimize over this set)
- Update the parameters: $\theta_t = \theta_{t-1} - \nu \frac{\partial \mathcal{L}_{B_t}}{\partial \theta}(\theta_{t-1})$
- $\frac{\partial \mathcal{L}_B}{\partial \theta}$ acts like a proxy for the (expensive, full) $\frac{\partial \mathcal{L}}{\partial \theta}$

SGD: Randomization Helps

- SGD vs Gradient Descent
  - Figure (a): Batch Gradient Descent ($B = N$)
  - Figure (b): SGD ($B < N$)
- If we initialize wrong, as in case 2-a below, standard Gradient Descent goes to a bad place
- Idea: Randomization helps and can allow the optimizer get to the right place even if we initialize wrong (i.e. 2-b)

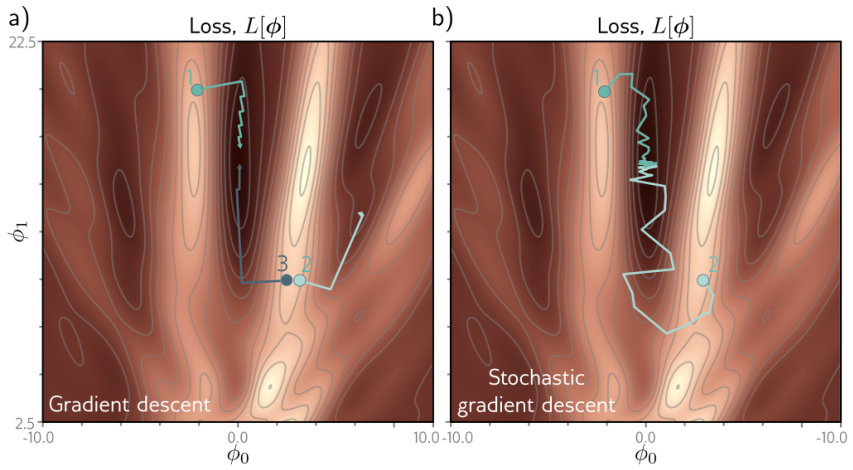


Figure 6.5 Gradient descent vs. stochastic gradient descent. a) Gradient descent with line search. As long as the gradient descent algorithm is initialized in the right "valley" of the loss function (e.g., points 1 and 3), the parameter estimate will move steadily toward the global minimum. However, if it is initialized outside this valley (e.g., point 2), it will descend toward one of the local minima. b) Stochastic gradient descent adds noise to the optimization process, so it is possible to escape from the wrong valley (e.g., point 2) and still reach the global minimum.

SGD: Batch size and noisy estimates

- Batches can be of size $B = 1$, 256, 128, etc.
- The smaller the batch, the noisier the estimate
- But, the smaller the batch, the more gradient updates (learning) you can do
- Since we compute $\frac{\partial \mathcal{L}_B}{\partial \theta}$ over batches, this is a noisy estimate for the gradient of $\frac{\partial \mathcal{L}}{\partial \theta}$ over the entire training set
- What are some methods that deal with this noise?

Momentum: Faster convergence with less variance in gradient updates

- We keep track of a momentum term $m_t = \beta m_{t-1} + (1-\beta)\frac{\partial \mathcal{L}_{B_t}}{\partial \theta}(\theta_{t-1})$
- The idea is $\frac{\partial \mathcal{L}_{B_t}}{\partial \theta}$ is choppy, but m_t is smoother
- The update would now be, for batch t, update via

$$\theta_t = \theta_{t-1} - \nu m_t$$

- Note m_t is a function of the past parameters $\theta_{t-1}, \theta_{t-2}, \ldots$
- As we progress through more and more iterations, we put less and less weight on older gradient estimates, but we never forget old gradients and updates are "slower and smoother"

Momentum

- Momentum adds smoothness to the loss trajectory
  - Figure (a): SGD (batch size B =1)
  - Figure (b): Momentum
- With momentum, the gradient estimate has less variance, we don't trash around in different directions
- The effective learning rate increases if all these gradients are aligned over multiple iterations but decreases if the gradient direction repeatedly changes as the terms in the sum cancel out

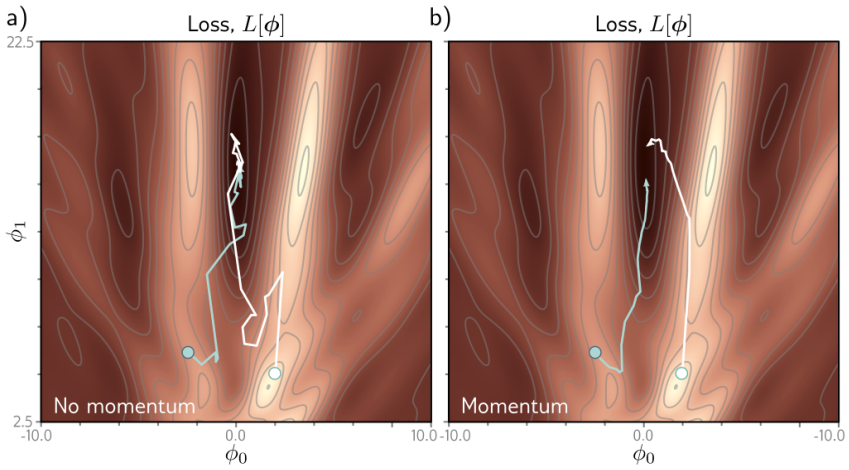


Figure 6.7 Stochastic gradient descent with momentum. a) Regular stochastic descent takes a very indirect path toward the minimum. b) With a momentum term, the change at the current step is a weighted combination of the previous change and the gradient computed from the batch. This smooths out the trajectory and increases the speed of convergence.

Nesterov Momentum: updating quicker

- The momentum term can be considered a coarse prediction of where the SGD algorithm will move next
- Nesterov accelerated momentum computes the gradient at this point rather than at the current point
- $m_t = \beta m_{t-1} + (1-\beta)\frac{\partial \mathcal{L}_{B_t}}{\partial \theta}(\theta_{t-1} - \nu m_{t-1})$
- Before, we used $m_t = \beta m_{t-1} + (1-\beta)\frac{\partial \mathcal{L}_{B_t}}{\partial \theta}(\theta_{t-1})$ whereas now we "guess" and evaluate there
- $$\theta_t = \theta_{t-1} - \nu m_t = \theta_{t-1} - \beta\nu m_{t-1} - (1-\beta)\nu\frac{\partial \mathcal{L}_{B_t}}{\partial \theta}(\theta_{t-1} - \nu m_{t-1})$$
- One way to think about this is that the gradient term now corrects the path provided by momentum alone

Nesterov Momentum

- Nesterov Momentum tries to pick the direction in a better way
  - Left: Momentum
  - Right: Nesterov Momentum
- Big idea: improve the rate of convergence by moving in a smarter direction

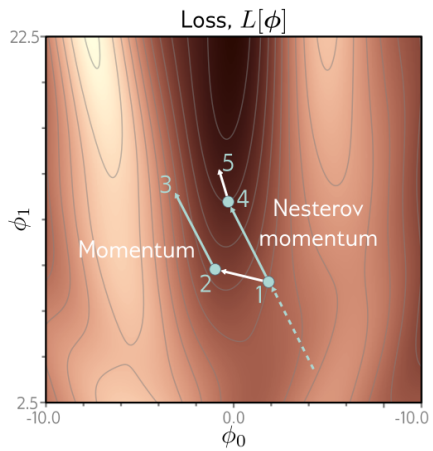


Figure 6.8 Nesterov accelerated momentum. The solution has traveled along the dashed line to arrive at point 1. A traditional momentum update measures the gradient at point 1, moves some distance in this direction to point 2, and then adds the momentum term from the previous iteration (i.e., in the same direction as the dashed line), arriving at point 3. The Nesterov momentum update first applies the momentum term (moving from point 1 to point 4) and then measures the gradient and applies an update to arrive at point 5.

Issues with SGD and normalization

- How do we make progress in both directions?
- Gradient descent with a fixed step size has the following undesirable property: it makes large adjustments to parameters associated with large gradients (where perhaps we should be more cautious) and small adjustments to parameters associated with small gradients (where perhaps we should explore further)
- When the gradient of the loss surface is much steeper in one direction than another, it is difficult to choose a learning rate that (i) makes good progress in both directions and (ii) is stable (figures 6.9a–b)

Issues with SGD

- Figure (a): Good progress vertically, slow horizontally
- Figure (b): Good progress horizontally, overshoot vertically

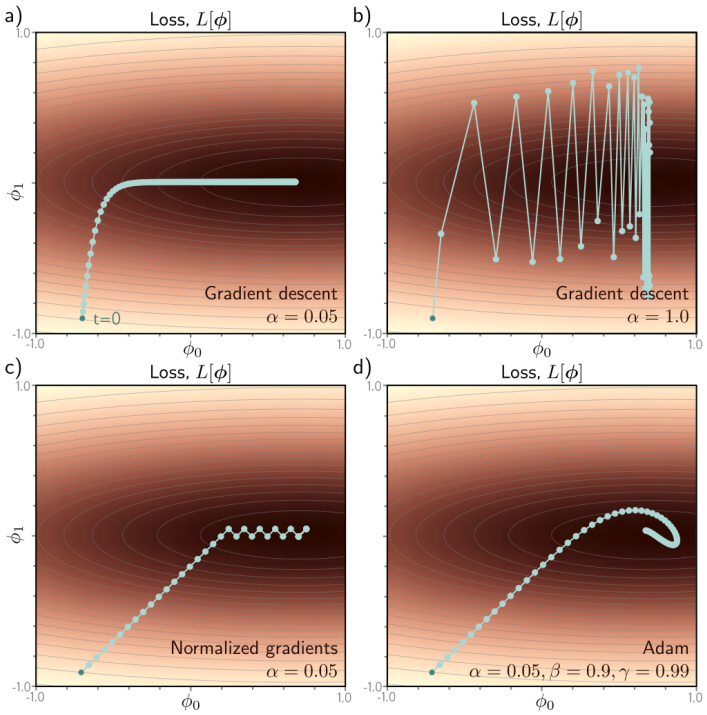


Figure 6.9 Adaptive moment estimation (Adam). a) This loss function changes quickly in the vertical direction but slowly in the horizontal direction. If we run full-batch gradient descent with a learning rate that makes good progress in the vertical direction, then the algorithm takes a long time to reach the final horizontal position. b) If the learning rate is chosen so that the algorithm makes good progress in the horizontal direction, it overshoots in the vertical direction and becomes unstable. c) A straightforward approach is to move a fixed distance along each axis at each step so that we move downhill in both directions. This is accomplished by normalizing the gradient magnitude and retaining only the sign. However, this does not usually converge to the exact minimum but instead oscillates back and forth around it (here between the last two points). d) The Adam algorithm uses momentum in both the estimated gradient and the normalization term, which creates a smoother path.

Towards Adam: Normalized Gradients

- Straightforward approach: is to normalize the gradients so that we move a fixed distance (governed by the learning rate) in each direction
- $m_t = \frac{\partial \mathcal{L}_{B_t}}{\partial \theta}(\theta_{t-1})$
- $v_t = (\frac{\partial \mathcal{L}_{B_t}}{\partial \theta}(\theta_{t-1}))^2$
- The term v_t is the point wise squared gradient, and the positive root of this is used to normalize the gradient itself, so all that remains is the sign in each coordinate direction
- $\theta_t = \theta_{t-1} - \nu \frac{m_t}{\sqrt{v_t}+\epsilon}$
- The result is that the algorithm moves a fixed distance ν along each coordinate, where the direction is determined by whichever way is downhill (see Figure (c) in the previous slide)
- This simple algorithm makes good progress in both directions
- But, it will not converge unless it happens to land exactly at the minimum
- Instead, it will bounce back and forth around the minimum

Issues with just scaling

- Figure (c): Normalized Gradients make the same conservative step size in any direction - but we need to differentiate!

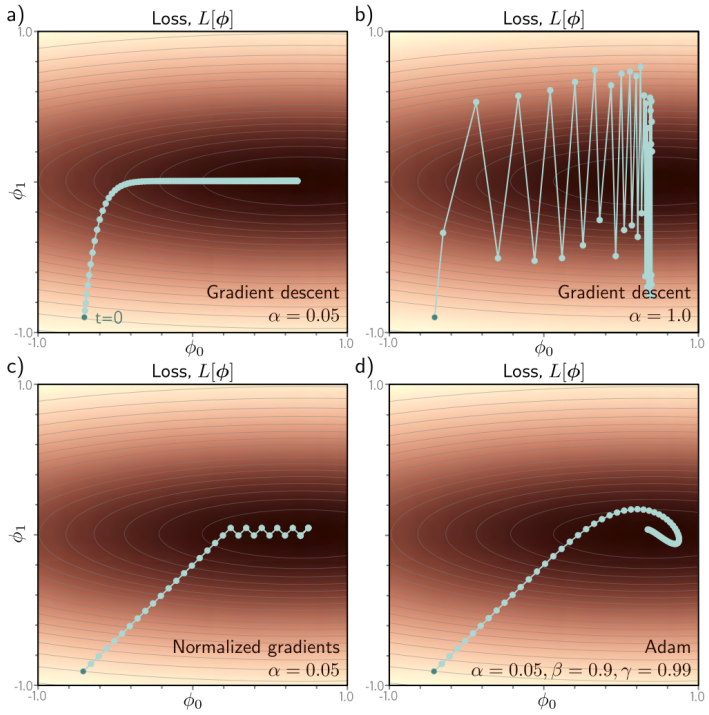


Figure 6.9 Adaptive moment estimation (Adam). a) This loss function changes quickly in the vertical direction but slowly in the horizontal direction. If we run full-batch gradient descent with a learning rate that makes good progress in the vertical direction, then the algorithm takes a long time to reach the final horizontal position. b) If the learning rate is chosen so that the algorithm makes good progress in the horizontal direction, it overshoots in the vertical direction and becomes unstable. c) A straightforward approach is to move a fixed distance along each axis at each step so that we move downhill in both directions. This is accomplished by normalizing the gradient magnitude and retaining only the sign. However, this does not usually converge to the exact minimum but instead oscillates back and forth around it (here between the last two points). d) The Adam algorithm uses momentum in both the estimated gradient and the normalization term, which creates a smoother path.

Adam

- Adam goes a step further, we also consider variance's momentum
- $m_t = (\beta_1 m_{t-1} + (1-\beta_1)\frac{\partial \mathcal{L}_{B_t}}{\partial \theta}(\theta_{t-1}))/(1-\beta_1^t)$
- We divide above by $(1-\beta_1^t)$ to have better estimates at the start, but this effect goes away as t gets larger
- $v_t = (\beta_2 v_{t-1} + (1-\beta_2)(\frac{\partial \mathcal{L}_{B_t}}{\partial \theta}(\theta_{t-1}))^2)/(1-\beta_2^t)$
- $\theta_t = \theta_{t-1} - \nu \frac{m_t}{\sqrt{v_t}+\epsilon}$
- The gradient magnitudes of neural network parameters can depend on their depth in the network
- Adam helps compensate for this tendency and balances out changes across the different layers
- In practice, Adam also has the advantage of being less sensitive to the initial learning rate because it avoids situations like those in Figures (a), (b) before, so it tends to not need complex learning rate schedules

Adam

- Figure (d): Adam (vs other methods)

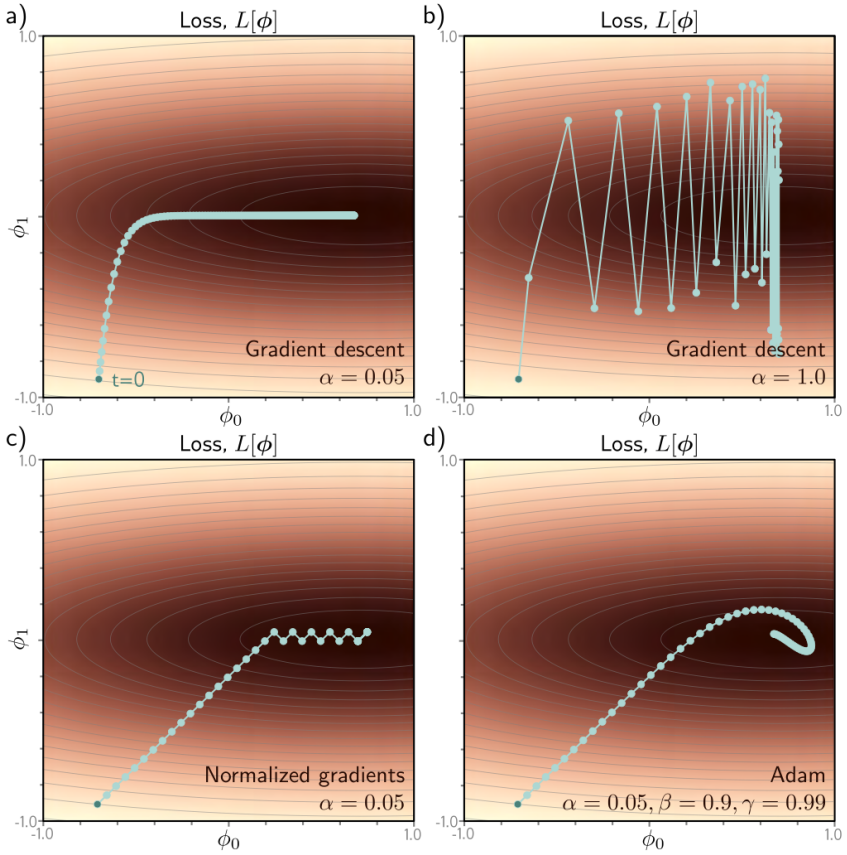


Figure 6.9 Adaptive moment estimation (Adam). a) This loss function changes quickly in the vertical direction but slowly in the horizontal direction. If we run full-batch gradient descent with a learning rate that makes good progress in the vertical direction, then the algorithm takes a long time to reach the final horizontal position. b) If the learning rate is chosen so that the algorithm makes good progress in the horizontal direction, it overshoots in the vertical direction and becomes unstable. c) A straightforward approach is to move a fixed distance along each axis at each step so that we move downhill in both directions. This is accomplished by normalizing the gradient magnitude and retaining only the sign. However, this does not usually converge to the exact minimum but instead oscillates back and forth around it (here between the last two points). d) The Adam algorithm uses momentum in both the estimated gradient and the normalization term, which creates a smoother path.

So just always use Adam?

- When it came out, people got excited with Adam and its performance
- After some time, people discovered that for some datasets Adam does not get to the optimal place, SGD is necessary
- The initial Adam paper had a mistake in its convergence proof
- How do you fix it? One idea was to run Adam and then switch to SGD after ... See the references
- Generally, Adam is a good optimizer to try, but you might need to do other things like the above

4 More Tricks

- Schedulers: Adjust the learning rate in a smart way
- Gradient Accumulation: How to use 1 GPU like you have 4
- Distributed Data Parallelism: One way to use 4 GPUs
- Warm Up: Don't just go from random parameters to better all at once, do it slowly so that the model is not "shocked"

Schedulers

- At a high level, batch gradient updates are of the sort

$$\theta_{new} = \theta_{old} - \nu \frac{\partial \mathcal{L}_{B_t}}{\partial \theta}(\theta_{old})$$

- This means that we can have

$$\frac{\theta_{new} - \theta_{old}}{\theta_{old}} = \nu \frac{\frac{\partial \mathcal{L}_{B_t}}{\partial \theta}(\theta_{old})}{\theta_{old}}$$

- Assume that $||\frac{\theta_{new} - \theta_{old}}{\theta_{old}}|| \sim \frac{1}{1000}$ so this is fairly stable
- This means that

$$-3 \sim \log_{10} \nu + \log_{10} \frac{std(\frac{\partial \mathcal{L}_{B_t}}{\partial \theta}(\theta_{old}))}{std(\theta_{old})}$$

- So this means
  - If $\log_{10} \nu + \log_{10} \frac{std(\frac{\partial \mathcal{L}_{B_t}}{\partial \theta}(\theta_{old}))}{std(\theta_{old})} < -3$, increase the learning rate
  - If $\log_{10} \nu + \log_{10} \frac{std(\frac{\partial \mathcal{L}_{B_t}}{\partial \theta}(\theta_{old}))}{std(\theta_{old})} > -3$, decrease the learning rate

Schedulers

- Ideas like the above are part of a larger theory of learning rate schedulers; the goal is to help learning when it stalls
- Popular schedulers in PyTorch such as ExponentialLR or ReduceLROnPlateau help change the learning rate as needed if some criterion is met
  - ExponentialLR: Every epoch, lower the learning rate by γ
  - ReduceLROnPlateau: If Val Accuracy has not gone lower for 3 iterations, lower the learning rate by γ
  - You can even make your own (i.e. the previous slide) ...

Gradient Accumulation

- Problem: you want to use a large batch size for optimization, but you have 1 GPU
- You find that for your model, you can fit just 2 training samples per GPU
- What can you do?
- You can, for example just use batch size 2 ... But each of these estimates are noisy

Large Batch Sizes

Key Benefits of Large Batch Sizes

- Computational Efficiency:
  - Parallelism: Larger batches allow for better utilization of hardware resources, especially GPUs which excel at parallel matrix multiplication and other operations
  - Reduced Communication Overhead: Your entire data is on the CPU and as you optimize you move it to the GPU (see above). The overhead of transferring data between the CPU and GPU is relatively fixed. With larger batches, the data transfer cost per example becomes lower
  - Enhanced Convergence: Large batch sizes allow for better utilization of parallel processing hardware (GPUs/TPUs), reducing the time taken per epoch

Large Batch Sizes

Key Benefits of Large Batch Sizes

- Improved Generalization?
  - Noisier Gradient Estimates: Smaller batches can introduce more noise into the gradient estimates and generalize better (See later) ; Batch Norm and learning rate warming can help mimic this even with large batch sizes
  - Reduced Variance: Larger batches can reduce the variance of the gradient estimates; this can lead to smoother and more consistent updates to the model parameters
- Larger Steps: Larger batches allow for taking larger steps in the optimization process. This can accelerate convergence, especially in the early stages of training
- One idea: start with a large batch size and then decrease it (LLAMA 3.1 used this)

Gradient Accumulation

- OK, so I want to use a large batch but I'm on Colab and I have 1 GPU
- One idea: Gradient Accumulation ...
  - Process K batches each of size B sequentially
  - For each batch, compute the loss gradient but do not do an update step
  - Instead, accumulate the K gradients by computing the gradient for a batch and sum with the previously computed gradient
  - Once you've processed K batches, then do a gradient step
  - Idea is now the gradient step is as if we've used a batch of size $B * K$

Gradient Accumulation

- Example: I want to use a batch size of 128 but I can only fit a batch size of 32 on my GPU (data + model)
  - Take 4 batches, each of size 32 and feed each one sequentially through the model and then get a gradient on each batch
  - Each loss gradient is a sum of error terms divided by 32, this is since a loss is the penalty summed across the 32 samples divided by $L = \frac{S}{32}$
  - We want the gradient of $\frac{S_1+S_2+S_3+S_4}{128}$ but we have the gradients $\frac{S_i}{32}$
  - To get what we want, multiply what we have by $\frac{32}{128}$
  - This is since

$$\frac{32}{128}(\frac{S_1}{32} + \ldots + \frac{S_4}{32}) = \frac{S_1 + S_2 + S_3 + S_4}{128}$$

What if you have multiple GPUs?

- What would I do if I had 4 GPUs and each could fit the model and a batch of maximum size 32?
- The idea now is something called Distributed Data Parallelism
  - Idea: make a copy of each model, one on each GPU
  - For a batch of size 128, split it into 4 chunks of size 32, and feed a batch of size 32 to each GPU
  - Compute the gradient of the model on each GPU - note the model is assumed to be the same on each GPU
  - Once we have the 4 computed gradients, accumulate them as before to get a gradient which effectively is of size 128
  - Put this gradient onto each GPU
  - Apply the same gradient step to each model on each GPU
  - Now, as before, each GPU has the same model but we've done a gradient step of size 128 effectively and not 32 ....
- Distributed Data Parallelism is a library in PyTorch you can leveredge

So do you always want to use a large batch?

- Rule of thumb: smaller batch sizes generalize better
- Large Batch sizes converge to sharp minima where while small batches converge to flat minima
- If small perturbations the optima vector cause a lot of data misclassification, the minimizer is sharp; a small movement away from the optimal parameters causes a large increase in the loss function. In contrast, flat minima have accuracy that remains nearly constant under small parameter perturbations

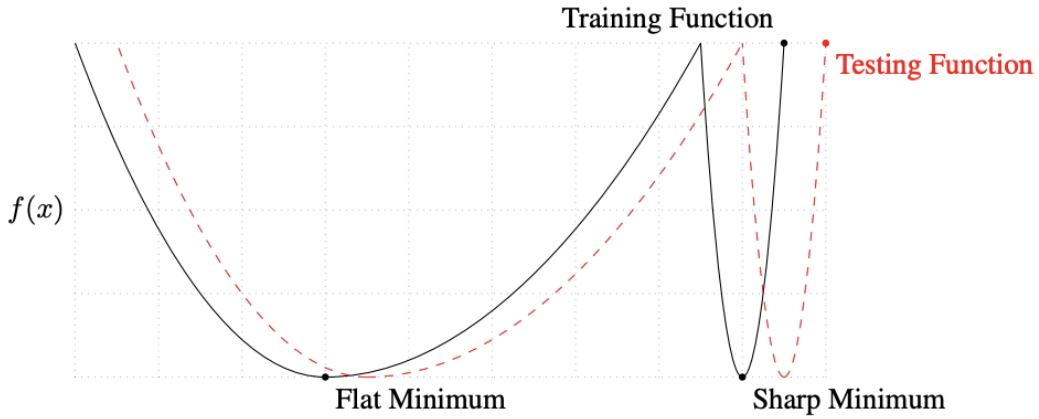
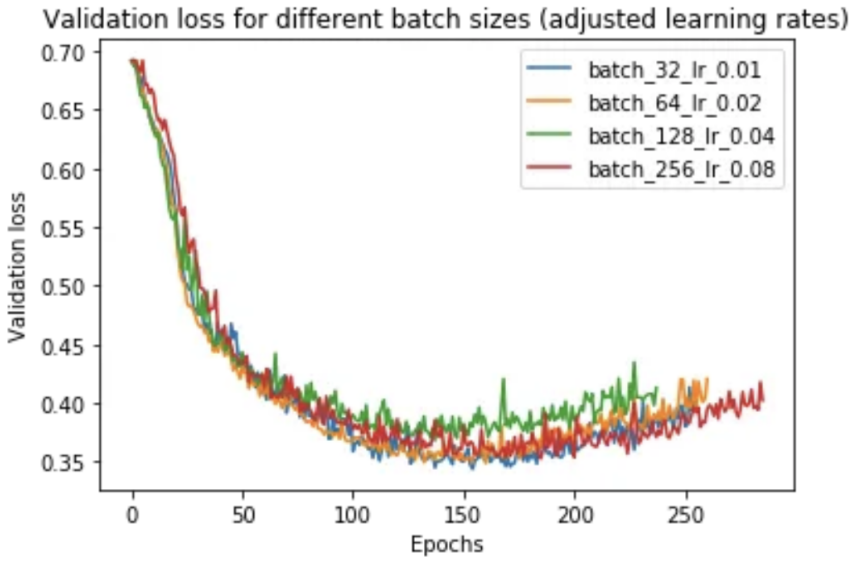


Figure 1: A Conceptual Sketch of Flat and Sharp Minima. The Y-axis indicates value of the loss function and the X-axis the variables (parameters)

So what do I do? Use small batch size?

- What to do?
- Generally speaking, you need to try out experiments with the batch size and see what happens
- There is no clean recipe always
- For example, people use learning rate warm up to get large batch training to still generalize well

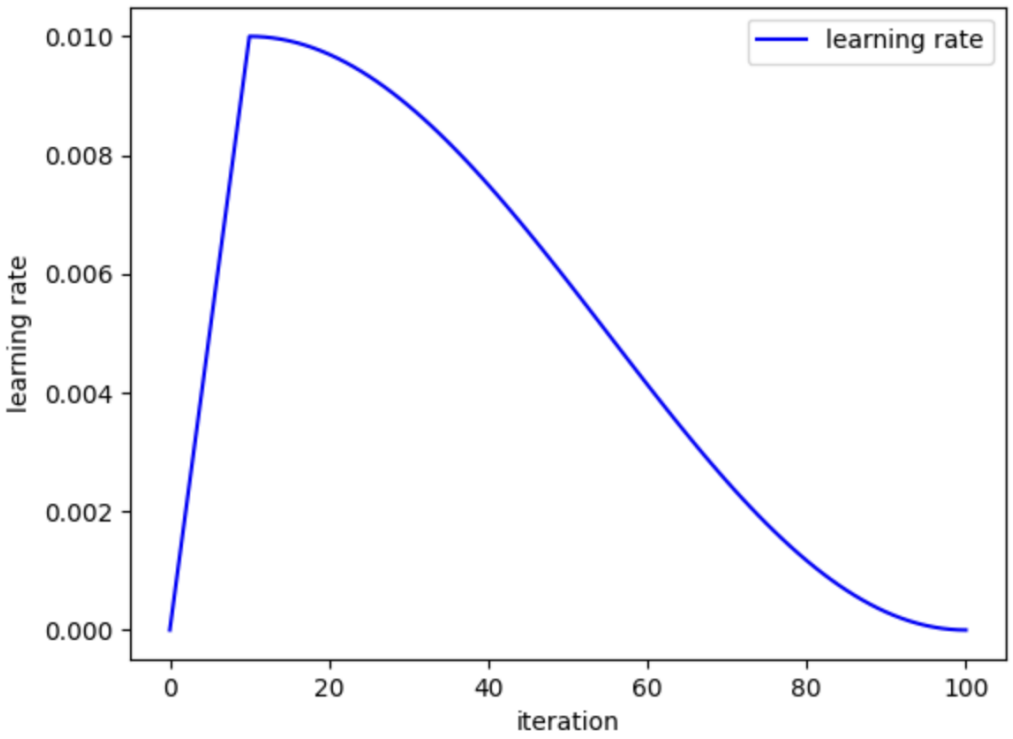


Notice both Batch Size and lr are increasing by 2 every time

Warm up?

- Idea is, we want to use a learning rate of 0.1
- However, we don't want this all at once
- Instead, increase the learning rate gradually from 0 to 0.1 in a few optimization epochs
- Once we get to 0.1, then dampen it
- Analytic Example: $lr = \alpha min\{\frac{1}{\sqrt{t}}, \frac{t}{\sqrt{10^3}}\}$
  - For $t \leq 10$ we use $\alpha \frac{t}{\sqrt{10^3}}$ (increase to α)
  - For $t > 10$ we use $\alpha \frac{1}{\sqrt{t}}$ (decrease once at α)

Warm up


learning rate
0.010
0.008
0.006
0.004
0.002
0.000
learning rate
0
20
40
60
80
100
iteration

Anything else about small batch size?

- From GoodFellow's Deep Learning (2016):
  - Small batches can offer a regularizing effect (Wilson and Martinez, 2003), perhaps due to the noise they add to the learning process
  - Generalization error is often best for a batch size of 1
  - Training with such a small batch size might require a small learning rate to maintain stability because of the high variance in the estimate of the gradient
  - The total runtime can be very high as a result of the need to make more steps, both because of the reduced learning rate and because it takes more steps to observe the entire training set

Outline

- Optimization - Theory + Examples
- Residual Connections
- BatchNorm and LayerNorm
- Regularization - Dropout

Vanishing Gradients

- Consider some Neural Network like below
  - $a^{[0]} = x$
  - $z^{[1]} = \beta^{[1]} a^{[0]} + \alpha^{[1]}$
  - $a^{[1]} = \sigma(z^{[1]})$
  - $z^{[2]} = \beta^{[2]} a^{[1]} + \alpha^{[2]}$
  - $a^{[2]} = \sigma(z^{[2]})$
  - $\ell = y \log a^{[2]} + (1 - y) \log 1 - a^{[2]}$
- In this example, we had $\frac{d\ell}{d\alpha^{[1]}} = (y - a^{[2]})\beta^{[2]}(a^{[1]}(1 - a^{[1]}))$
- This is the same as $(y - a^{[2]})\beta^{[2]}\sigma(z^{[1]})(1 - \sigma(z^{[1]}))$
- What happens if $z^{[1]}$ is large?

What if $z^{[1]}$ is small / large?

- If $z^{[1]}$ is small, $\sigma(z^{[1]}) = 0$
- If $z^{[1]}$ is large, $\sigma(z^{[1]}) = 1$
- In either case, we have $\sigma(z^{[1]})(1 - \sigma(z^{[1]})) = 0$ so that $\frac{d\ell}{d\alpha^{[1]}} = 0$
- This is bad, since now there is no learning for $\alpha^{[1]}$ as we have

$$\alpha^{[1]} = \alpha^{[1]} + \nu \frac{d\ell}{d\alpha^{[1]}} = \alpha^{[1]}$$

and we stay in the same place

Residual Connections

- What if we modify the last few computations to be
  - $a^{[0]} = x$
  - $z^{[1]} = \beta^{[1]} a^{[0]} + \alpha^{[1]}$
  - $a^{[1]} = \sigma(z^{[1]})$
  - $z^{[2]} = \beta^{[2]} a^{[1]} + \alpha^{[2]} + z^{[1]}$
  - $a^{[2]} = \sigma(z^{[2]})$
  - $\ell = y \log a^{[2]} + (1 - y) \log 1 - a^{[2]}$
- Then we have $\frac{d\ell}{d\alpha^{[1]}} = \frac{d\ell}{da^{[2]}} \frac{da^{[2]}}{dz^{[2]}} \frac{dz^{[2]}}{da^{[1]}} \frac{da^{[1]}}{dz^{[1]}} \frac{dz^{[1]}}{d\alpha^{[1]}} + \frac{d\ell}{da^{[2]}} \frac{da^{[2]}}{dz^{[2]}} \frac{dz^{[2]}}{dz^{[1]}} \frac{dz^{[1]}}{d\alpha^{[1]}}$
- Why? We can go
  - $\ell \to a^{[2]} \to z^{[2]} \to a^{[1]} \to z^{[1]} \to \alpha^{[1]}$
  - $\ell \to a^{[2]} \to z^{[2]} \to z^{[1]} \to \alpha^{[1]}$
- The extra term above is $(y - a^{[2]})$ which is nonzero even if $z^{[1]}$ is large or small; we have less of a chance of something going wrong

Highway Connections

- What if we modify the last few computations to be
  - $a^{[0]} = x$
  - $z^{[1]} = \beta^{[1]} a^{[0]} + \alpha^{[1]}$
  - $a^{[1]} = \sigma(\theta^{[1]} a^{[0]} + \vartheta^{[1]})\sigma(z^{[1]}) + (1 - \sigma(\theta^{[1]} a^{[0]} + \vartheta^{[1]}))a^{[0]}$
  - $z^{[2]} = \beta^{[2]} a^{[1]} + \alpha^{[2]}$
  - $a^{[2]} = \sigma(\theta^{[2]} a^{[1]} + \vartheta^{[2]})\sigma(z^{[2]}) + (1 - \sigma(\theta^{[2]} a^{[1]} + \vartheta^{[2]}))a^{[1]}$
  - $\ell = y \log a^{[2]} + (1 - y) \log 1 - a^{[2]}$
- The idea here is that we have many more paths than even before
- For example, when considering its output $a^{[2]}$, the Network can learn either the identity (and so output $a^{[1]}$) or the mapping $\sigma(z^{[2]})$

Outline

- Optimization - Theory + Examples
- Residual Connections
- BatchNorm and LayerNorm
- Regularization - Dropout

BatchNorm

- We optimize a Neural Network over batches
- At a high level, for each element in a batch we feed it into a nonlinearity, so we get $y = \sigma(x)$
- What if x is large or small?
- As before, we have that if x is large or small, $\sigma(x)$ will be 0 or 1, and this might affect the gradient of parameters in lower layers
- This is since $\frac{d\sigma(x)}{dx} = \sigma(x)(1 - \sigma(x))$

BatchNorm

- We optimize a Network over batches and batches can change
- A batch might change distribution, so for example x feeding into σ might be all over the place
- The loss will be very bouncy
- Idea: Normalize the data before feeding into the nonlinearity!

BatchNorm

- For each batch $\mathcal{B}$, compute $\mu_{\mathcal{B}}$ and $\sigma^2_{\mathcal{B}}$ which are the mean and variance of the different components of x in the batch $\mathcal{B}$ (these are vectors, of the same dimension as x)
- For example, if $\mathcal{B} = \{x_1, \ldots, x_m\}$ and each $x_i \in \mathbb{R}^d$, then so are $\mu_{\mathcal{B}}$ and $\sigma^2_{\mathcal{B}}$ but each component of these vectors is the mean / variance of the respective components of all batch vectors x_b, i.e. $(\mu_{\mathcal{B}})(j) = \frac{\sum_{i=1}^m x_i(j)}{m}$
- Before we feed into σ, normalize each vector $x \in \mathcal{B}$ so that we use $\hat{x} = \frac{x - \mu_{\mathcal{B}}}{\sqrt{\sigma^2_{\mathcal{B}} + \epsilon}}$
- Now, feed this normalized $\hat{x}$ into σ: less chance we'll get extremes!
- But, what if we want to allow the network to learn the scaling and off-centering?
- Introduce two (vector-valued) parameters γ, β
- Use $\gamma\hat{x} + \beta$ as the input to σ
- Maybe $\gamma = 1$ and $\beta = 0$, but let the network learn that

BatchNorm

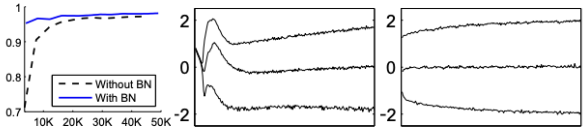
- So the process goes as follows
- Fix a batch $\mathcal{B} = \{x_1, \ldots, x_m\}$ and compute $\mu_{\mathcal{B}} = mean(x_{\mathcal{B}})$ and $\sigma_{\mathcal{B}} = stdev(x_{\mathcal{B}})$

$$
\begin{aligned}
\mu_{\mathcal{B}} &= mean(x_{\mathcal{B}}) \\
\sigma_{\mathcal{B}} &= stdev(x_{\mathcal{B}}) \\
\hat{x} &= \frac{x - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} \\
y &= \gamma \hat{x} + \beta
\end{aligned}
$$

- Feed y to a nonlinearity
- Basically, before you run the data through the nonlinearity, apply a batch normalization!

BatchNorm: Benefits

- Higher learning rates!
- Stability!
- When training with Batch Normalization, a training example is seen in conjunction with other examples in the mini-batch, and the training network no longer producing deterministic values for a given training example
- This last point might lead to a bit of regularization



(a) (b) Without BN (c) With BN

Figure 1: (a) *The test accuracy of the MNIST network trained with and without Batch Normalization, vs. the number of training steps. Batch Normalization helps the network train faster and achieve higher accuracy.* (b, c) *The evolution of input distributions to a typical sigmoid, over the course of training, shown as* $\{15, 50, 85\}$*th percentiles. Batch Normalization makes the distribution more stable and reduces the internal covariate shift.*

BatchNorm: Gradients

- BatchNorm gradients are not that scary: everything is differentiable!
- Here, ℓ is the loss, and we have simplifications like

$$\frac{\partial \ell}{\partial \hat{x}_i} = \frac{\partial \ell}{\partial y_i}\frac{\partial y_i}{\partial \hat{x}_i} = \frac{\partial \ell}{\partial y_i}\gamma$$

During training we need to backpropagate the gradient of loss ℓ through this transformation, as well as compute the gradients with respect to the parameters of the BN transform. We use chain rule, as follows (before simplification):

$$\frac{\partial \ell}{\partial \widehat{x}_i} = \frac{\partial \ell}{\partial y_i} \cdot \gamma$$

$$\frac{\partial \ell}{\partial \sigma_{\mathcal{B}}^2} = \sum_{i=1}^m \frac{\partial \ell}{\partial \widehat{x}_i} \cdot (x_i - \mu_{\mathcal{B}}) \cdot \frac{-1}{2}(\sigma_{\mathcal{B}}^2 + \epsilon)^{-3/2}$$

$$\frac{\partial \ell}{\partial \mu_{\mathcal{B}}} = \left(\sum_{i=1}^m \frac{\partial \ell}{\partial \widehat{x}_i} \cdot \frac{-1}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}}\right) + \frac{\partial \ell}{\partial \sigma_{\mathcal{B}}^2} \cdot \frac{\sum_{i=1}^m -2(x_i - \mu_{\mathcal{B}})}{m}$$

$$\frac{\partial \ell}{\partial x_i} = \frac{\partial \ell}{\partial \widehat{x}_i} \cdot \frac{1}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} + \frac{\partial \ell}{\partial \sigma_{\mathcal{B}}^2} \cdot \frac{2(x_i - \mu_{\mathcal{B}})}{m} + \frac{\partial \ell}{\partial \mu_{\mathcal{B}}} \cdot \frac{1}{m}$$

$$\frac{\partial \ell}{\partial \gamma} = \sum_{i=1}^m \frac{\partial \ell}{\partial y_i} \cdot \widehat{x}_i$$

$$\frac{\partial \ell}{\partial \beta} = \sum_{i=1}^m \frac{\partial \ell}{\partial y_i}$$

BatchNorm: Training and Inference

- At training time, we compute $\mu_{\mathcal{B}}$ and $\sigma^2_{\mathcal{B}}$ for each batch and use these statistics in gradient computations
- At inference time, we can recompute these statistics over the entire training set and use these as normalizations
- Or, we can use some sort of running estimate (momentum!)
- For example, if we process mini-batches $\mathcal{B}_1, \mathcal{B}_2, \ldots, \mathcal{B}_T$ in sequence, we can define a mean estimate as $\mu_{\mathcal{B}} = \mu_{\mathcal{B}_1}$, then $\mu_{\mathcal{B}} = \alpha\mu_{\mathcal{B}_2} + (1-\alpha)\mu_{\mathcal{B}}$ etc
- We can use $\mu_{\mathcal{B}}$ at inference time
- Same for $\sigma^2_{\mathcal{B}}$
- **Be careful**: the behavior of this layer depends on what state your model is in - if it is in eval or train mode!

LayerNorm

- For NLP applications, usually we have batches of sentences and each sentence has different tokens and different lengths
- Each token has an embedding
- We can't really take a mean across all embeddings in the batch, since different embeddings are in different sentences
- We'd lose context if we did this (imagine we have to pad short sentences - those 0's would affect the means)
- For NLP, a batch is of dimension $B \times T \times d$
- Before, a batch was of dimension $B \times d$ or $B \times H \times W \times C$ and the statistics were of dimension d or $H \times W \times C$
- For NLP, it does not make sense to get statistics across $T \times d$ since we go between sentences which can be very different

LayerNorm

- A better way is to normalize each word's (or token's) embedding
- Thus, μ_d and σ_d^2 are the mean and variance across all components of a particular token embedding (these are scalars and we use broadcasting to do the calculations below)
- This method does not depend on the batch size
- We normalize $\hat{e} = \frac{e-\mu_d}{\sqrt{\sigma_d^2+\epsilon}}$
- Finally, we apply γ and β as before (again, these are vectors) to allow the model to learn the transformation
- We use $\gamma\hat{e} + \beta$ as the final representation of LayerNorm

Outline

- Optimization - Theory + Examples
- Residual Connections
- BatchNorm and LayerNorm
- Regularization - Dropout

Drop Out

- The aim of dropout is to make sure the model does not overtrain too much
- At a high level, for each batch we zero out a random proportion (20 %, 30 %) of the neurons
- This effectively zeros out the parameters for these neurons
- For a certain weight W, we effectively use $D * W$ where D is a Bernoulli random variable that is $1/0$ with probability $1 - p$ and p

Drop Out

- You can think of this method kind of like ensembling: on each batch, you are training a slightly different model that is weakened vs the full model

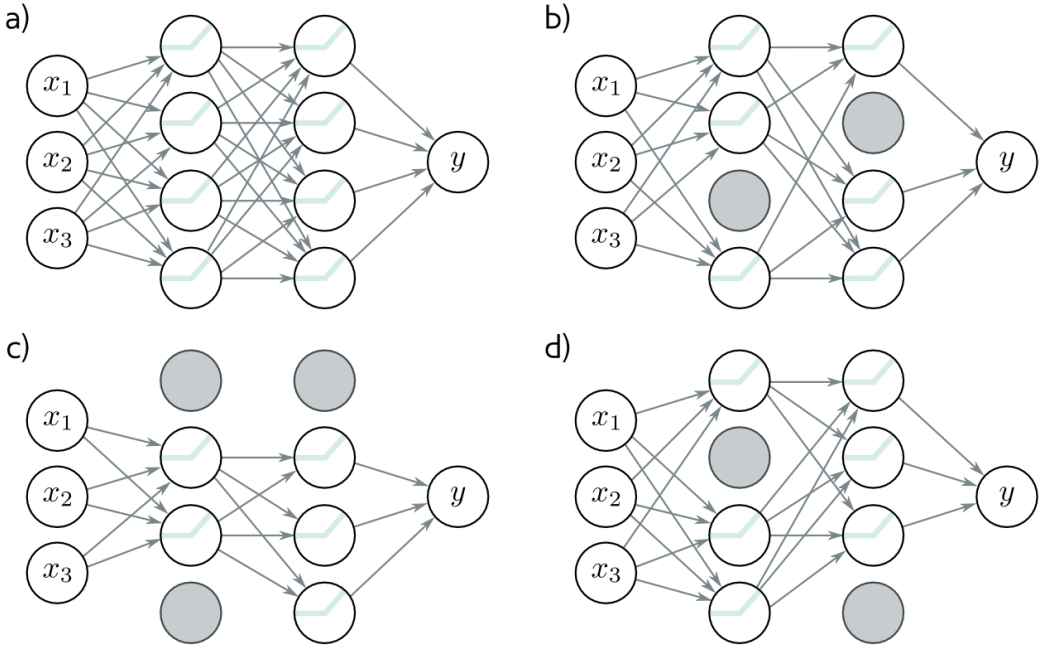


Figure 9.8 Dropout. a) Original network. b-d) At each training iteration, a random subset of hidden units is clamped to zero (gray nodes). The result is that the incoming and outgoing weights from these units have no effect and so we are training with a slightly different network each time.

Drop Out

- You can also think dropout as making the model learn as much as it can in this weakened state: do the best with what you have, don't get too comfortable using one set of nodes
- This makes the network less dependent on any given hidden unit
- We also encourage the weights to have smaller magnitudes so that the change in the function due to the presence or absence of the hidden unit is reduced

Drop Out

- **Be careful**: the behavior of this layer (like Batch Norm) depends on what state (train or eval) your model is in!
- Specifically, at inference time for a weight W, we use

$$E[D * W] = 1 * P(D = 1) * W + 0 * P(D = 0) * W = (1 - p)W$$

  as the weight of the layer
- At training time, we use $D * W$
- See References: L_2 regularization can be viewed as a sister of dropout; specifically, dropout can be seen as a type of generalized L_2 regularization

References

- Neural Machine Translation
- Understanding Deep Learning
- A Neural Probabilistic Language Model
- Res Net
- Batch Norm
- Layer Norm
- Highway Networks
- What Batch size to use?
- Switch to SGD from Adam
- Dropout and L_2
- Batch Size Paper, 2003
- Deep Learning 2016 Book
- On Large-Batch Training For Deep Learning: Generalization Gap and Sharp Minima